

POINTPILOT: UNIFIED ROBOTIC MANIPULATION VIA OBJECT-CENTRIC 3D POINT FLOW PREDICTION

Anonymous authors

Paper under double-blind review

ABSTRACT

Generalization remains a central challenge in robotics, calling for policies that bridge differences from multiple dimensions such as diverse embodiments, camera poses, or scene layouts. In order to approach this problem, we introduce **PointPilot**, a 3D aligned and task-focused manipulation framework built on a 3D space point-flow prediction. Rather than modeling the global scene, PointPilot only represents task-relevant objects (robot grippers treated as objects), avoiding the prediction of potentially redundant scene parts unrelated to the manipulation tasks. Different from previous works in this direction, PointPilot aligns 3D spatial representation in one unified coordinate and supports direct action execution from object point flow without a separately trained action expert or a dedicated controller for establishing an initial grasping. Given sensor observations and language instructions, our model jointly learns subtask planning, task-relevant objects grounding, and 3D point flow prediction in an end-to-end way. By processing heterogeneous data in a shared 3D point flow space, PointPilot is able to absorb demonstrations from different embodiments (include human hands). Extensive simulative and real world experiments validate our method. *Notably*, we demonstrate the zero-shot embodiment generalization in the PointPilot—one trained policy can be deployed onto an previously unseen embodiment and successfully complete manipulation tasks, without any additional fine-tuning.

1 INTRODUCTION

A central promise of generalist robot learning is that manipulation policies should improve as they absorb increasingly diverse data. However, realizing this promise requires more than scaling model parameters and demonstration hours. Robot data is heterogeneous not only in visual appearance, task semantics, and environment configuration, but also in embodiments. Native actions from heterogeneous data lack a shared representation form, preventing action prediction from directly serving as a unified supervision space. The choice of prediction space therefore determines whether heterogeneous demonstrations provide compatible supervision or remain isolated by their control interfaces.

Vision–language–action(VLA) models have made substantial progress toward pretraining a single policy across tasks and robot platforms (Open X-Embodiment Collaboration, 2024; Ghosh et al., 2024; Black et al., 2025) and finetuning with a specific task. Their apparent unification primarily occurs at the model level, for the reason that learned targets remain robot actions, whose dimensionality, coordinate convention, and control semantics depend on the embodiment and manipulation setup. Existing systems accommodate these differences through data pre-processing, embodiment conditioning, or specialized action heads (Open X-Embodiment Collaboration, 2024; Wang et al., 2024; Doshi et al., 2025; Zheng et al., 2025; 2026).

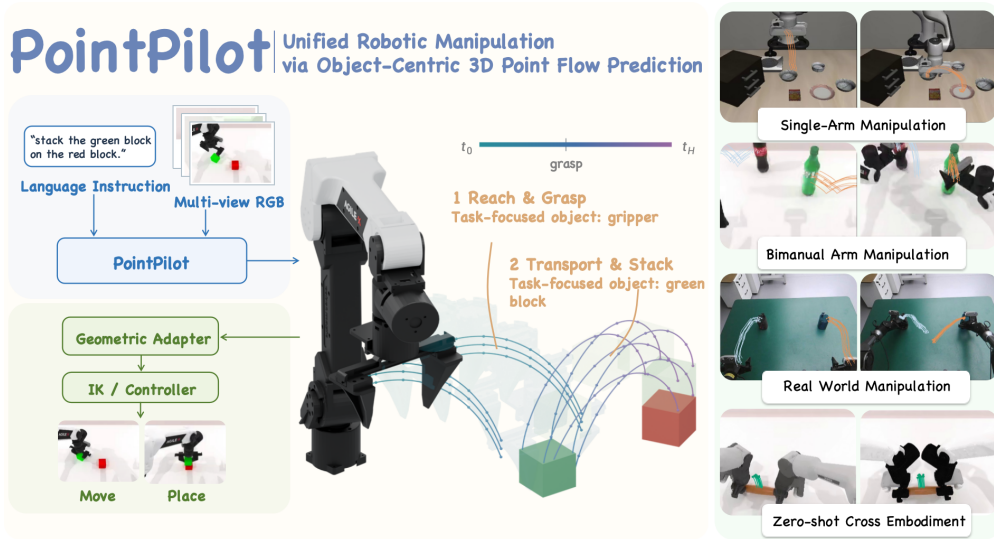


Figure 1: **Overview of PointPilot.** **Left:** Given a language instruction and multi-view RGB observations, PointPilot predicts task-focused objects’ 3D point flow in each subtask. A non-learned geometric adapter converts the predicted flow into robot end-effector pose commands for execution. **Right:** PointPilot unifies in different manipulation modes and embodiments, thus supporting single-arm and bimanual gripper to execute any manipulation tasks. We further demonstrate real-world manipulation and zero-shot cross-embodiment skill transfer.

Task-relevant object motion offers a natural source of shared supervision because it captures task motion without committing to a robot’s joints or control conventions. This insight has motivated a growing family of representations, including articulation flow, general 3D object flow, object-part scene flow, dense 3D motion fields, language-conditioned point trajectories, and point-cloud world models (Eisner et al., 2022; Yuan et al., 2025; Xu et al., 2025; Tang et al., 2025; Yin et al., 2025; Zhang et al., 2026; Huang et al., 2025b; Lin et al., 2026).

Despite of the great advances these approaches made in the object-centric prediction for robotic manipulation, a unified and scalable framework is still hard to achieve. To our best knowledge, existing approaches generally fall into three patterns: **a)** they use object motion as an intermediate latent feature for an additional embodiment-dependent action decoder—reverting to a VLA-style action prediction paradigm (Xu et al., 2025; Yang et al., 2025; Lin et al., 2026); **b)** depend on manual manipulation or an extra controller to grasp objects at the beginning (Tang et al., 2025; Yin et al., 2025; Yuan et al., 2025); **c)** or specialize object-centric representations to one particular manipulation mode (Eisner et al., 2022; Lin et al., 2026; Tang et al., 2025; Yang et al., 2025; Yuan et al., 2025). **In particular**, manipulation modes such as single-arm or bimanual arm manipulation may not be a problem in VLA paradigm, but it is a problem in the object-centric prediction. Unlike VLA, different manipulation modes require different numbers of objects flow and different object-to-gripper motion assignments. To simplify this, most object-centric methods specify their flow to one object and only execute single arm tasks.

As far as we are concerned, a unified object-centric prediction framework should be **manipulation-mode agnostic**, allowing a single model to handle both single-arm and dual-arm manipulation by predicting motion for a variable number of objects; **embodiment-agnostic**, allowing heterogeneous robot demonstrations to be composed despite incompatible native action spaces; and **directly executable**, allowing the object flow prediction to control the robot in a close loop without an additional embodiment-dependent action decoder. Existing object-centric approaches typically satisfy only a subset of these requirements, leaving object motion either mode-specific, embodiment-dependent, or disconnected from execution.

To alleviate this gap, we decompose robotic manipulation into subtasks and objectify each substage in a shared 3D space. Particularly, **1)** we define the *task-focused objects* for point flow learning. Unlike some object-centric models which are confined to one specific manipulation mode, this simple design allows one or multiple object flows to represent different manipulation modes within one

model. Moreover, **2)** we align input geometry and predicted motion in a shared coordinate frame anchored to the reference camera. By expressing all views in this frame, we establish cross-view spatial correspondences that facilitate the fusion of visual and geometric information. Additionally, **3)** we directly use object flow to guide the task execution without an additional learned and embodiment dependent action expert. Thus, our model can transfer skills to multiple previously unseen embodiments and achieve zero-shot cross-embodiment generalization. Finally, **4)** integrating these characteristics, we novelly propose *PointPilot*, a closed-loop and fully autonomous object-centric 3D point flow model that unifies in 3D space, manipulation modes and embodiments. PointPilot connects high-level task planning and objects grounding with low-level point-flow forecasting and robot action execution.

2 RELATED WORK

Table 1: **Comparison with prior object-centric methods.** Cross-embodiment transfer denotes deployment across robot embodiments without additional training of the motion prediction model. No separate grasping module requires the model neither has a manual grasp initialization nor a separate grasping module. No learned motion-to-action head requires the predicted flow can be directly executed by the EEf without a post-training action head. $\times$ denotes the method neither has nor demonstrate this ability in the paper. * denotes transfer between different end effectors on the same arm. RoboFlow4D is not a typical object-centric method, but it considers the gripper as an object, so we also include it as a baseline.

Method	Zero-shot cross-embodiment transfer	No separate grasping module	No learned motion-to-action head	Single-arm tasks	Bimanual tasks
Im2Flow2Act (Xu et al., 2025)	$\times$	$\checkmark$	$\times$	$\checkmark$	$\times$
PPI (Yang et al., 2025)	$\times$	$\checkmark$	$\times$	$\times$	$\checkmark$
RoboFlow4D (Lin et al., 2026)	$\times$	$\checkmark$	$\times$	$\checkmark$	$\times$
General Flow (Yuan et al., 2025)	$\times$	$\times$	$\checkmark$	$\checkmark$	$\times$
Object-Part Scene Flow (Tang et al., 2025)	$\checkmark^*$	$\times$	$\checkmark$	$\checkmark$	$\times$
3D Motion Field (Yin et al., 2025)	$\times$	$\times$	$\checkmark$	$\checkmark$	$\times$
FlowBot3D (Eisner et al., 2022)	$\times$	$\checkmark$	$\checkmark$	$\checkmark$	$\times$
ImaginationPolicy (Lu et al., 2025)	$\times$	$\checkmark$	$\checkmark$	$\checkmark$	$\times$
PointPilot (Ours)	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$

2.1 OBJECT-CENTRIC MOTION REPRESENTATIONS FOR MANIPULATION

Prior works have shown that object-centric 3D flow provides a better generalization across unseen objects, scenes, and robot embodiments (Eisner et al., 2022; Tang et al., 2025; Yuan et al., 2025; Yin et al., 2025).

However, existing object-centric approaches broadly follow several patterns. One line treats predicted flow or point trajectories as intermediate motion guidance for downstream action expert training (Xu et al., 2025; Yang et al., 2025; Lin et al., 2026; Zhang et al., 2026). One line directly converts predicted objects’ flow into robot trajectories, but does not provide a complete autonomous and close-looped manipulation. They either manually manipulate the gripper or use an extra controller to grasp the object. (Bharadhwaj et al., 2024; Yuan et al., 2025; Zhi et al., 2025). Another line also often confined to one specific manipulation mode, such as the single-arm manipulation (Eisner et al., 2022; Yin et al., 2025; Yuan et al., 2025; Zhi et al., 2025; Lu et al., 2025; Lin et al., 2026). Most works only execute single-arm tasks for the reason that only one object needs to be predicted. The detailed comparison is shown in Table 1.

3 TASK-FOCUSED OBJECTS FOR 3D POINT FLOW LEARNING

We define a *task-focused object* as a physical entity whose motion characterizes the manipulation required at the current stage and can be mapped to the motion of one or both grippers. Specifically, for each gripper, the task-focused object is either the gripper itself or a locally rigid object part currently coupled to it. To support different manipulation modes, each task-focused object is additionally assigned an *execution role*, indicating the gripper to which its predicted point flow should be mapped.

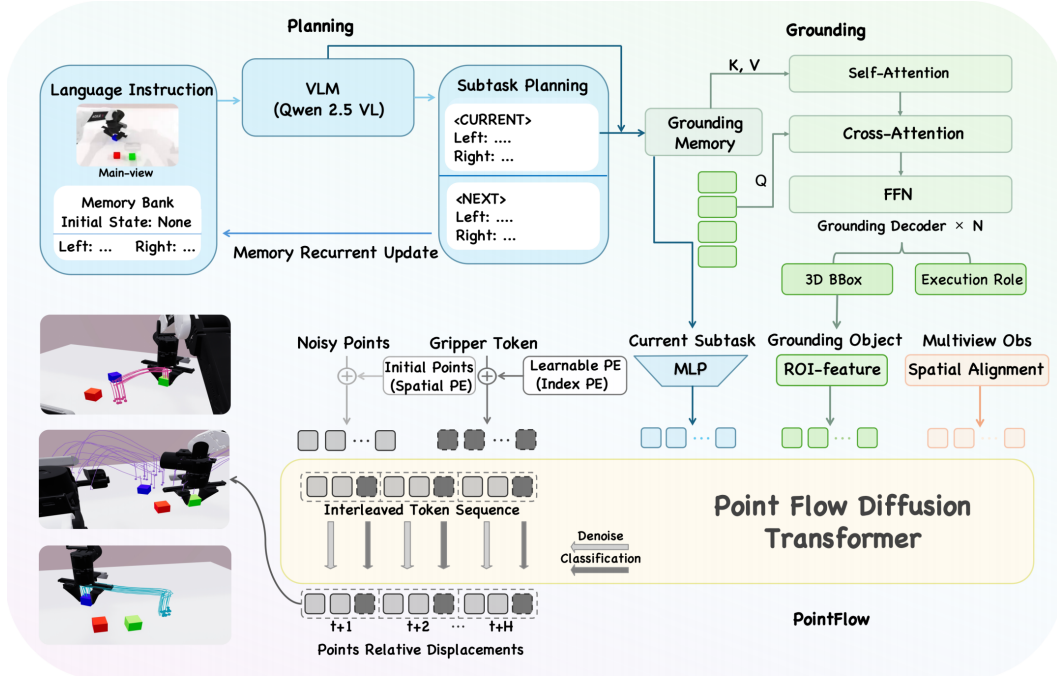


Figure 2: **Method of PointPilot.** (a) **Planning:** A memory-conditioned VLM predicts current and next subtasks. (b) **Grounding:** Task-conditioned 3D grounding localizes task-focused objects and assigns execution roles. (c) **PointFlow:** Multi-view observations are geometrically aligned and used by Point DiT to jointly predict relative 3D point flow and gripper states. Predicted object motion is converted to end-effector trajectories through learning-free geometry adapter for closed-loop execution.

For example, when the left gripper approaches an object, the left gripper itself serves as the task-focused object with the execution role *Left*. Once the object is grasped and transported, the task-focused object switches from the gripper to the coupled object, whose motion then directly characterizes the desired manipulation.

Under this formulation, PointPilot represents both gripper motion and manipulated-object motion within a shared point-flow space by casting manipulation as the motion of task-focused objects.

4 POINTPILOT: UNIFIED OBJECT-CENTRIC 3D POINT FLOW PREDICTION

Building on the definition of *task-focused objects*, we introduce *PointPilot*. In this section, we will present the model architecture, covering subtask planning, object grounding, 3D geometric alignment, cross-view fusion, and point-flow predicting, and then describe how the predicted point flow is mapped to executable end-effector trajectories through a learning-free adapter.

4.1 SUBTASK PLANNING

We use a memory-conditioned VLM planner to identify the current manipulation stage and anticipate the next subtask. At planning time t , the planner receives the main-view RGB image I_t^{main} , the global language instruction ℓ , and a textual memory bank $\mathcal{M}_t$ that records previously predicted subtasks. At the beginning of each episode, the memory bank is initialized to NONE.

Using Qwen2.5-VL as the backbone, the planner generates a structured plan containing CURRENT and NEXT spans:

$$(\hat{s}_t, \hat{s}_t^+) = \text{VLM}_\theta(I_t^{\text{main}}, \ell, \mathcal{M}_t). \quad (1)$$

Each span specifies a left/right subtask pair, with NONE indicating an inactive hand. The CURRENT span $\hat{s}_t$ describes the subtasks to execute at the current stage. The NEXT span $\hat{s}_t^+$ specifies the immediate next distinct subtask pair expected to begin within the H -step horizon, or NONE if no

new next subtask is expected. The hidden states associated with `CURRENT` provide task features $\mathbf{Z}_t^{\text{task}}$ for subsequent 3D object grounding and point-flow prediction.

For the next planning call at $t + H$, the memory is updated using only the previously predicted subtask spans:

$$\mathcal{M}_{t+H} = \text{Concat}(\mathcal{M}_t, \hat{s}_t, \hat{s}_t^+). \quad (2)$$

The detailed design of VLM prompt and memory bank is indicated in the Appendix.

4.2 LANGUAGE-GUIDED 3D GROUNDING

Geometry Frustum Inspired by PETR in autonomous driving and DexBEV in embodied intelligence, we construct the discretized camera frustums for frames. In details, we first inject camera-aware 3D positional information into the main-view image tokens. For an image token at $\tilde{\mathbf{u}}_i = [u_i, v_i, 1]^\top$, we sample D depth hypotheses along its corresponding camera ray using linearly increasing discretization (LID). Let N_{img} denote the number of main-view image tokens and D the number of depth hypotheses. For $i \in \{1, \dots, N_{\text{img}}\}$ and $k \in \{0, \dots, D - 1\}$, we define

$$z_k = z_{\min} + \frac{z_{\max} - z_{\min}}{D(D+1)} k(k+1) \in [z_{\min}, z_{\max}), \quad \mathbf{x}_{i,k}^c = z_k \tilde{\mathbf{K}}^{-1} \tilde{\mathbf{u}}_i \in \mathbb{R}^3. \quad (3)$$

where $\tilde{\mathbf{K}} \in \mathbb{R}^{3 \times 3}$ is the main-camera intrinsic matrix, and c denotes the main-camera coordinate frame. Following these equations, we obtain a ray-aligned 3D coordinate tensor $\mathbf{X}_i^c \in \mathbb{R}^{D \times 3}$ for each main-view image token. Collectively, these coordinates form a discretized camera frustum $\mathbf{X}^c \in \mathbb{R}^{N_{\text{img}} \times D \times 3}$.

The resulting camera-frame coordinates are then encoded into a 3D positional embedding $\mathbf{e}_i^{3D} \in \mathbb{R}^C$, which is added to the corresponding final-layer VLM image feature $\mathbf{H}_{t,i}^{\text{img}}$, obtaining d -dimensional geometry-aware image tokens. This construction requires neither observed nor predicted depth.

Grounding & Classification We concatenate these geometry-aware image tokens with the planner’s current-subtask token features $\mathbf{Z}_t^{\text{task}}$ to form the grounding memory. A six-layer transformer decoder updates Q learnable queries through self-attention and cross-attention to this memory. Each decoded query predicts a 3D box with center $\mathbf{c} \in \mathbb{R}^3$, dimensions $\boldsymbol{\eta} \in \mathbb{R}^3$, and a 6D rotation representation $\mathbf{r} \in \mathbb{R}^6$, along with an execution role from $\{\text{invalid}, \text{Left}, \text{Right}, \text{Left\&Right}\}$.

The execution role associates each task-focused object with the gripper(s) responsible for its manipulation: Left routes this object’s point flow to the left gripper, and the Left&Right denotes that both grippers are coupled to the same object and follow its motion.

During training, we use Hungarian matching to establish a one-to-one assignment between predicted queries and ground-truth task-focused objects. Matched queries are supervised with their assigned 3D boxes and execution roles, while unmatched queries are assigned the invalid role and excluded from box regression. The matching cost and loss definitions are detailed in the appendix.

4.3 POINT FLOW DIT

Spatial Alignment We use the main-view coordinate as the anchored reference frame for both observations and point flow prediction. Auxiliary-view images are encoded by the shared vision encoder. Following Sec. 4.2, we construct their discretized camera frustums and transform the sampled 3D coordinates into the main-camera frame using calibrated camera extrinsics:

$$\tilde{\mathbf{x}}_{i,k}^c = \mathbf{T}_{c_v \rightarrow c} \tilde{\mathbf{x}}_{i,k}^{c_v}, \quad v \in \mathcal{V}_{\text{aux}} \quad (4)$$

where c_v denotes the coordinate frame of auxiliary views v , and $\mathbf{T}_{c_v \rightarrow c}$ is the coordinate transformation from auxiliary views v to the main view. These aligned frustums provide 3D positional embeddings for the auxiliary-view visual features.

Point Flow Prediction For each grounded object, we use farthest point sampling (FPS) within its predicted 3D bounding box to obtain N initial points. PointPilot predicts their future displacements $\Delta \mathbf{p}_h = \mathbf{p}_h - \mathbf{p}_0$ over an H -step horizon.

To augment the condition of the grounded object, each grounded object is represented by an object token $\mathbf{Z}_t^{\text{obj}}$. This token fuses geometry features of its sampled initial points processed by a light

PointNet with grounded visual appearance features extracted by Region of Interest Align (ROIAlign) from the projected 3D bounding box region.

For point-flow generation, each future point displacement is represented by a noisy point token. In order to preserve its spatial identity, we augment each token with a spatial position embedding of the corresponding current 3D anchor point in the main-camera coordinate frame. Therefore, the model predicts motion from spatially grounded point tokens rather than from pure noise. Furthermore, at each future step, noisy point tokens are interleaved with learned gripper tokens, enabling joint prediction of object motion and gripper states. We train the DiT with conditional flow matching to predict the velocity field that transports noisy point tokens to future point displacements. Gripper tokens are not noised and directly predict binary open/close states without the noising-denoising process.

The Point DiT is conditioned on the grounding features, object tokens, and spatially aligned auxiliary-view visual features.

4.4 FROM POINT FLOW TO EXECUTABLE ROBOT ACTIONS

Before grasping, predicted gripper flow guides execution. After grasping, object flow guides the coupled gripper, assuming a rigid grasp. Our learning-free adapter transforms current and predicted points into the world frame and fits a rigid motion:

$$(\hat{\mathbf{R}}_{t,h}^m, \hat{\mathbf{d}}_{t,h}^m) = \arg \min_{\mathbf{R} \in \text{SO}(3), \mathbf{d} \in \mathbb{R}^3} \sum_{n=1}^N \left\| \mathbf{R} \mathbf{p}_{t,n}^{m,w} + \mathbf{d} - \hat{\mathbf{p}}_{t+h,n}^{m,w} \right\|_2^2. \quad (5)$$

The fitted relative transform $\Delta \hat{\mathbf{T}}$ updates each assigned end effector’s current pose $\mathbf{T}_a$ to its target pose $\hat{\mathbf{T}}_a = \Delta \hat{\mathbf{T}} \mathbf{T}_a$, with all poses expressed in the world frame. For Left&Right, this transform acts on both grippers’ current poses; independent motions use separate object flows. The resulting poses and predicted gripper states are executed through IK and the robot controller. PointPilot observes the updated scene and replans after each action chunk.

4.5 TRAINING OBJECTIVE

PointPilot is jointly optimized for subtask planning, 3D grounding, point-flow prediction, and gripper control:

$$\mathcal{L} = \lambda_1 \mathcal{L}_{\text{plan}} + \lambda_2 \mathcal{L}_{\text{ground}} + \lambda_3 \mathcal{L}_{\text{pointflow}} + \lambda_4 \mathcal{L}_{\text{gripper}}. \quad (6)$$

The VLM is trained with cross entropy in a teacher forcing way. Grounding combines 3D box regression and execution-role classification under Hungarian matching. Conditioned on predicted bounding boxes, point-flow prediction uses conditional flow matching on anchor-relative displacements, and gripper states are supervised with binary cross entropy. Full loss definitions, matching costs, masking rules, and optimization settings are provided in the appendix.

5 DATA PIPELINE FOR TASK-FOCUSED OBJECTS POINT FLOW

We construct PointPilot’s stage-level supervision from simulated and real-robot demonstrations through interaction-stage segmentation and 3D trajectory extraction. This aligns task semantics with task-focused object motion without requiring shared low-level action encodings across data sources.

5.1 STAGE SEGMENTATION

We primarily segment demonstrations using gripper open/close events. In simulation, gripper states and grasp relationships define interaction boundaries; for real-robot data, the same events are detected from the smoothed gripper-width signal. Contact and articulated-part motion cues are used when gripper events are insufficient. Bimanual interactions are aligned on a shared timeline, and offline stage-level language annotations are matched to the resulting segments.

5.2 TRAJECTORY EXTRACTION

Simulation. We directly retrieve EEF and object pose trajectories from simulator states. For articulated interactions, we track the manipulated moving part rather than the full object assembly. All trajectories are transformed into a consistent reference frame and aligned with the segmented stages.

Real-world. EEF trajectories are obtained directly from recorded robot poses. For grasped-object stages, we recover object geometry once at a visual anchor and propagate it using the recorded EEF motion, avoiding frame-wise tracking. Following the visual-grounding pipeline of MolmoMotion (Zhang et al., 2026), we combine MolmoPoint localization (Clark et al., 2026) with SAM 3 masks (Carion et al., 2025), and lift the selected region to 3D using estimated depth and camera calibration.

Let T_B^A denote the transform from frame B to frame A , and let a be the anchor time. Assuming rigid object–gripper coupling, the object pose during the grasp interval is reconstructed as

$$\hat{T}_{O_t}^C = T_{E_t}^C (T_{E_a}^C)^{-1} \hat{T}_{O_a}^C, \quad t \in \mathcal{I}_{\text{coupled}}. \quad (7)$$

Here, $\hat{T}_{O_a}^C$ is the visually estimated anchor pose, while recorded EEF motion provides its temporal propagation. Frames outside the accepted coupling interval are not extrapolated.

Appendix A details validity checks and failure handling for depth calibration, anchor selection, mask filtering, and reconstruction. We align stage intervals, language annotations, and valid trajectories by their original timestamps to construct sparse point-flow supervision.

6 EXPERIMENTS

We evaluate PointPilot from three perspectives: (1) general manipulation performance on both single-arm, dual-arm simulation and real world tasks, (2) zero-shot skill transfer across previously unseen robot embodiments, and (3) the controlled modular ablations to indicate the efficiency of each modules especially the necessity of Spatial alignment.

6.1 SIMULATION RESULT

Training and evaluation setup. PointPilot is jointly trained on demonstrations from LIBERO and RoboTwin, combining different robot embodiments and single-arm and bimanual manipulation within one model. The results on both benchmarks therefore evaluate a shared policy learned from heterogeneous demonstrations. We follow the standard evaluation protocols of LIBERO and RoboTwin 2.0 Clean and report task success rates. Our observation configurations and evaluation details are provided in the appendix.

Single-arm manipulation on LIBERO. As shown in table 2, PointPilot achieves 85.8% average success across the four LIBERO suites, including the best listed results on LIBERO-Spatial (96.0%) and LIBERO-Object (97.6%). It obtains 77.2% on LIBERO-Long. Performance on LIBERO-Goal is mainly limited by three non-coupled, friction-driven tasks; excluding them, success reaches 96.0%. These results demonstrate competitive single-arm manipulation using object-centric point flow without a learned end-effector action decoder.

Table 2: Success rates (%) on LIBERO.

Method	Spatial	Object	Goal	Long	Average
Octo (Ghosh et al., 2024)	78.9	85.7	84.6	51.1	75.1
CogACT (Li et al., 2024)	87.5	90.2	78.4	53.2	77.3
OpenVLA (Kim et al., 2025)	84.7	88.4	79.2	53.7	76.5
TraceVLA (Zheng et al., 2024)	84.6	85.2	75.1	54.1	74.8
SpatialVLA (Qu et al., 2025)	88.2	89.9	78.6	55.5	78.1
4D-VLA (Zhang et al., 2025)	88.9	95.2	90.9	79.1	88.6
DP w/ RoboFlow4D (Lin et al., 2026)	89.8	93.2	85.2	72.0	85.1
DiT w/ RoboFlow4D (Lin et al., 2026)	90.2	97.0	88.4	75.2	87.7
UniAct (ACT head) (Zheng et al., 2025)	77.0	87.0	77.0	70.0	77.8
SmolVLA (2.25B) (Shukor et al., 2025)	93.0	94.0	91.0	77.0	88.8
PointPilot (Ours)	96.0	97.6	72.4	77.2	85.8

Baseline results follow the evaluation reported by RoboFlow4D (Lin et al., 2026).

Table 3: Success rates (%) on RoboTwin 2.0.

Method	Success rate (%)
DP	28.0
ACT	29.7
RDT-1B	34.5
GR00T-N1.7	43.6
π_0	46.4
DP3	55.2
PointPilot (Ours)	56.6

Baseline results are reported by RoboTwin 2.0 (Chen et al., 2025). [†]The GR00T-N1.7 result is from the Easy setting of RoboTwin-Clean2Rand reported by Qwen-RobotManip (Qwen Team, 2026).

Dual-arm manipulation on RoboTwin. The same jointly trained model achieves 56.6% success on RoboTwin 2.0 Clean, compared with the reported 55.2% of DP3. Together with the LIBERO results, this demonstrates that a shared object-centric policy can learn from heterogeneous demonstrations and support both single-arm and dual-arm manipulation.

6.2 ZERO-SHOT CROSS-EMBODIMENT TRANSFER

We evaluate zero-shot transfer from the ALOHA embodiment in RoboTwin to ARX and Piper on the tasks in Table 4. The jointly trained policy is deployed without target-embodiment demonstrations or fine-tuning. Predicted object motion is converted into executable commands through the geometric adapter and each target robot’s kinematics and controller.

Table 4: Cross-embodiment success rates (%) on eight tasks (10 trials per evaluated task). For each embodiment, the experiment seeds are the same. The only difference is the embodiment. X-VLA is tested with its official checkpoint not trained on ARX and Piper. In robotwin, not all tasks are valid in all embodiments. “–” indicates this task is incompatible with this embodiment. * denotes the average and Δ are calculated across 7 tasks; the alarm-clock task was not evaluated.

Task	ALOHA		ARX		Piper	
	PointPilot	X-VLA	PointPilot	X-VLA	PointPilot	X-VLA
Grab roller	100.0	100.0	90.0	80.0	100.0	20.0
Shake bottle	100.0	100.0	90.0	80.0	100.0	30.0
Shake bottle horizontally	100.0	100.0	90.0	90.0	100.0	30.0
Click alarm clock	70.0	0.0	60.0	60.0	–	–
Click bell	80.0	100.0	70.0	100.0	70.0	0.0
Dump bin into big bin	100.0	100.0	70.0	10.0	60.0	40.0
Stack two bowls	90.0	30.0	80.0	10.0	60.0	0.0
Pick dual bottles	50.0	100.0	40.0	90.0	40.0	50.0
Average	86.3	78.8	73.8	65.0	75.7*	24.3*
Δ compared with ALOHA (pp)	–	–	-12.5	–13.8	-12.9*	–65.7*

As shown in Table 4, we compare PointPilot with X-VLA on zero-shot transfer from ALOHA to ARX and Piper. Without target-embodiment demonstrations or fine-tuning, PointPilot achieves higher average success rates on both target embodiments and exhibits a smaller transfer gap, particularly on Piper.

6.3 REAL WORLD RESULTS

We evaluate PointPilot on three real-world tasks: retail barcode scanning, pouring from a bottle, and placing a Rubik’s cube into a bin, with 20 trials per task. PointPilot operates autonomously in a closed loop, replanning from updated observations after each action chunk. Table 5 reports success rates, and Fig. 3 illustrates staged execution. Rollout videos are provided in the supplementary material.

6.4 ABLATION STUDIES

We ablate two key components of PointPilot: the planning memory bank and the 3D geometric alignment used for point-flow prediction.

Effect of planning memory. As shown in Tab. 6, removing the memory bank consistently degrades performance across all LIBERO suites, with the largest drop on LIBERO-Long (77.2% $\rightarrow$ 55.0%). Without planning history, the VLM frequently repeats an already executed subtask during consecutive replanning steps, causing the policy to stall. The memory bank therefore mainly improves temporal consistency in long-horizon manipulation.

Effect of 3D geometry and multi-view alignment. Fig. 4 studies the contribution of geometric and multi-view information to point-flow learning. Adding the main-view 3D frustum reduces the

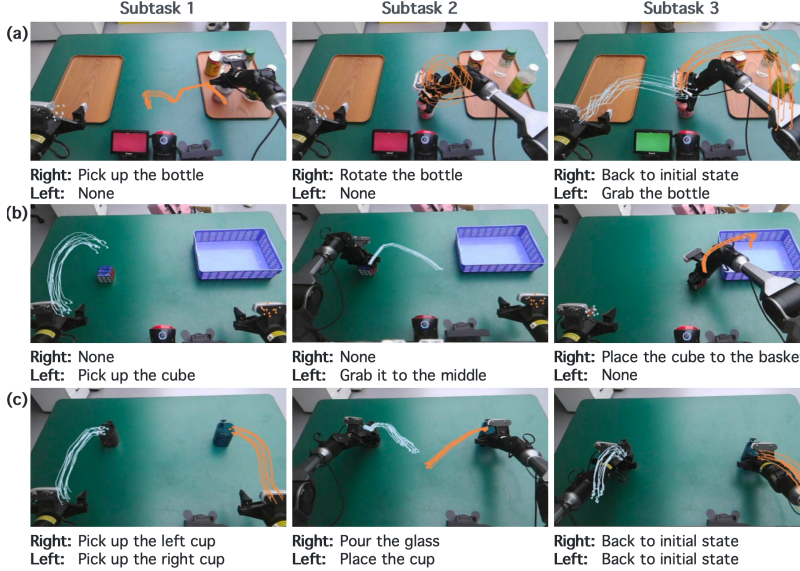


Figure 3: Real-world rollouts of PointPilot. Each row shows three subtasks from left to right: (a) retail barcode scanning, (b) placing a Rubik’s cube in a bin, and (c) pouring from a bottle.

Table 5: Real-world success rates.(over 20 trials)

(a) Retail barcode scanning	
Trials	20
Successes	10
Rate (%)	50
(b) Placing a Rubik’s cube in a bin	
Trials	20
Successes	18
Rate (%)	90
(c) Pouring from a bottle	
Trials	20
Successes	14
Rate (%)	70
Average (%)	70

Table 6: Effect of the planning memory bank on LIBERO success rates. Memory consistently improves performance across all four suites, with the largest gain on LIBERO-Long (+22.2 points)

Suite	w/o Memory	PointPilot	Δ
Spatial	90.4	96.0	+5.6
Object	89.6	97.6	+8.0
Goal	65.9	72.4	+6.5
Long	55.0	77.2	+22.2
Average	75.2	85.8	+10.6

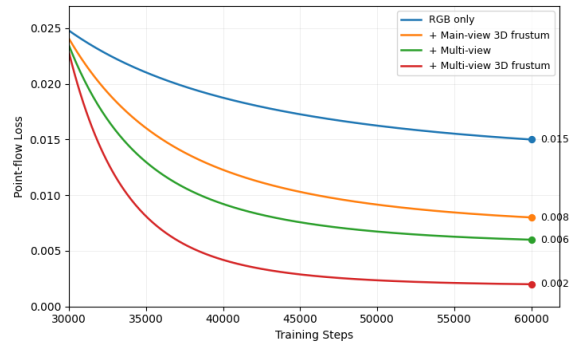


Figure 4: Point-flow loss under different visual-geometric configurations. Main-view 3D frustum encoding, additional views, and multi-view 3D frustum alignment progressively improve convergence.

final point-flow loss from 0.015 to 0.008. Introducing auxiliary views further lowers it to 0.006, while applying 3D aligned frustum encoding to all views achieves the lowest loss of 0.002. These results indicate that both multi-view observations and explicit 3D geometric alignment substantially facilitate point-flow prediction.

7 CONCLUSION AND FUTURE WORK

PointPilot represents stage-dependent task-focused objects with sparse 3D points and execution roles, providing a shared interface across supported manipulation modes and embodiments. Memory-conditioned subtask planning and grounding connect task intent to point-flow prediction, with multi-view geometry and predicted motion aligned in the main-camera frame. Under stage-local rigid coupling, a non-learned geometric adapter converts predicted flows into end-effector motion without an additional learned continuous-action decoder.

The current framework focuses on locally rigid task-focused objects and gripper-based execution. Future directions include dexterous hands, human hand-object demonstrations, and larger-scale real-robot data, with hand articulation and changing contacts requiring further study. Modeling additional task-relevant entities could capture broader object-object interactions and indirect effects. Reducing the burden of aligning stage semantics, object geometry, execution roles, and motion while preserving semantic-geometric consistency remains a challenge for scaling.

AI USE STATEMENT

(This section is **required** and does not count toward the page limit.)

In this work, we used generative AI tools for [tasks with required disclosure]. We have not used generative AI tools for [other tasks with required disclosure], and [the rest of the required disclosure tasks] are not applicable to this work. Additionally, we used generative AI tools for [tasks with recommended disclosure]. We have reviewed all AI-assisted work. [Elaborate. For example, “we checked LLM-generated research ideas for potential plagiarism through a manual literature survey”, “LLM-generated code was verified and tested for correctness by 2 authors”, etc.]. We take responsibility for the final content of this work, including text, claims or artifacts produced with the aid of generative AI.

See the ICLR 2027 AI Policy for Authors for more details. This statement should not be more than 1 page.

ETHICS STATEMENT

(This section is **recommended** and does not count toward the page limit.)

If authors feel that their paper submission raises questions regarding the Code of Ethics, they are encouraged to include a paragraph of Ethics Statement (at the end of the main text before references) to address potential concerns where appropriate. Topics include, but are not limited to, studies that involve human subjects, practices to data set releases, potentially harmful insights, methodologies and applications, potential conflicts of interest and sponsorship, discrimination/bias/fairness concerns, privacy and security issues, legal compliance, and research integrity issues (e.g., IRB, documentation, research ethics). This statement should not be more than 1 page.

REPRODUCIBILITY STATEMENT

(This section is **recommended** and does not count toward the page limit.)

It is important that the work published in ICLR is reproducible. Authors are strongly encouraged to include a paragraph-long Reproducibility Statement at the end of the main text (before references) to discuss the efforts that have been made to ensure reproducibility. This paragraph should not itself describe details needed for reproducing the results, but rather reference the parts of the main paper, appendix, and supplemental materials that will help with reproducibility. For example, for novel models or algorithms, a link to an anonymous downloadable source code can be submitted as supplementary materials; for theoretical results, clear explanations of any assumptions and a complete

proof of the claims can be included in the appendix; for any datasets used in the experiments, a complete description of the data processing steps can be provided in the supplementary materials. Each of the above are examples of things that can be referenced in the reproducibility statement.

AUTHOR CONTRIBUTIONS

If you'd like to, you may include a section for author contributions as is done in many journals. This is optional and at the discretion of the authors.

ACKNOWLEDGMENTS

Use unnumbered third level headings for the acknowledgments. All acknowledgments, including those to funding agencies, go at the end of the paper.

REFERENCES

- Homanga Bharadhwaj, Roozbeh Mottaghi, Abhinav Gupta, and Shubham Tulsiani. Track2Act: Predicting point tracks from internet videos enables generalizable robot manipulation. In *European Conference on Computer Vision (ECCV)*, pp. 306–324, 2024. doi: 10.1007/978-3-031-73116-7_18. URL https://doi.org/10.1007/978-3-031-73116-7_18.
- Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Robert Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, Szymon Jakubczak, Tim Jones, Liyiming Ke, Sergey Levine, Adrian Li-Bell, Mohith Mothukuri, Suraj Nair, Karl Pertsch, Lucy Xiaoyang Shi, Laura Smith, James Tanner, Quan Vuong, Anna Walling, Haohuan Wang, and Ury Zhilinsky. π_0 : A Vision-Language-Action flow model for general robot control. In *Proceedings of Robotics: Science and Systems*, Los Angeles, CA, USA, June 2025. doi: 10.15607/RSS.2025.XXI.010. URL <https://doi.org/10.15607/RSS.2025.XXI.010>.
- Nicolas Carion, Laura Gustafson, Yuan-Ting Hu, Shoubhik Debnath, Ronghang Hu, Didac Suris, Chaitanya Ryali, Kalyan Vasudev Alwala, Haitham Khedr, Andrew Huang, Jie Lei, Tengyu Ma, Baishan Guo, Arpit Kalla, Markus Marks, Joseph Greer, Meng Wang, Peize Sun, Roman Rädle, Triantafyllos Afouras, Effrosyni Mavroudi, Katherine Xu, Tsung-Han Wu, Yu Zhou, Liliane Momeni, Rishi Hazra, Shuangrui Ding, Sagar Vaze, Francois Porcher, Feng Li, Siyuan Li, Aishwarya Kamath, Ho Kei Cheng, Piotr Dollár, Nikhila Ravi, Kate Saenko, Pengchuan Zhang, and Christoph Feichtenhofer. SAM 3: Segment anything with concepts. *arXiv preprint arXiv:2511.16719*, 2025. URL <https://arxiv.org/abs/2511.16719>.
- Tianxing Chen, Zanzin Chen, Baijun Chen, Zijian Cai, Yibin Liu, Zixuan Li, Qiwei Liang, Xianliang Lin, Yiheng Ge, Zhenyu Gu, et al. Robotwin 2.0: A scalable data generator and benchmark with strong domain randomization for robust bimanual robotic manipulation. *arXiv preprint arXiv:2506.18088*, 2025.
- Christopher Clark, Yue Yang, Jae Sung Park, Zixian Ma, Jieyu Zhang, Rohun Tripathi, Mohamadreza Salehi, Sangho Lee, Taira Anderson, Winson Han, and Ranjay Krishna. MolmoPoint: Better pointing for VLMs with grounding tokens. *arXiv preprint arXiv:2603.28069*, 2026. URL <https://arxiv.org/abs/2603.28069>.
- Ria Doshi, Homer Rich Walke, Oier Mees, Sudeep Dasari, and Sergey Levine. Scaling cross-embodied learning: One policy for manipulation, navigation, locomotion and aviation. In *Proceedings of the 8th Conference on Robot Learning*, volume 270 of *Proceedings of Machine Learning Research*, pp. 496–512. PMLR, 2025. URL <https://proceedings.mlr.press/v270/doshi25a.html>.
- Ben Eisner, Harry Zhang, and David Held. FlowBot3D: Learning 3D articulation flow to manipulate articulated objects. In *Proceedings of Robotics: Science and Systems*, New York City, NY, USA, June 2022. doi: 10.15607/RSS.2022.XVIII.018. URL <https://doi.org/10.15607/RSS.2022.XVIII.018>.
- Dibya Ghosh, Homer Rich Walke, Karl Pertsch, Kevin Black, Oier Mees, Sudeep Dasari, Joey Hejna, Tobias Kreiman, Charles Xu, Jianlan Luo, You Liang Tan, Lawrence Yunliang Chen,

- Quan Vuong, Ted Xiao, Pannag R. Sanketi, Dorsa Sadigh, Chelsea Finn, and Sergey Levine. Octo: An open-source generalist robot policy. In *Proceedings of Robotics: Science and Systems*, Delft, Netherlands, July 2024. doi: 10.15607/RSS.2024.XX.090. URL <https://doi.org/10.15607/RSS.2024.XX.090>.
- Jiahui Huang, Qunjie Zhou, Hesam Rabeti, Aleksandr Korovko, Huan Ling, Xuanchi Ren, Tianchang Shen, Jun Gao, Dmitry Slepichev, Chen-Hsuan Lin, Jiawei Ren, Kevin Xie, Joydeep Biswas, Laura Leal-Taixe, and Sanja Fidler. ViPE: Video pose engine for 3D geometric perception, 2025a. URL <https://arxiv.org/abs/2508.10934>.
- Suning Huang, Qianzhong Chen, Xiaohan Zhang, Jiankai Sun, and Mac Schwager. ParticleFormer: A 3D point cloud world model for multi-object, multi-material robotic manipulation. In *Proceedings of the 9th Conference on Robot Learning*, volume 305 of *Proceedings of Machine Learning Research*, pp. 4941–4957. PMLR, 2025b. URL <https://proceedings.mlr.press/v305/huang25c.html>.
- Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan P. Foster, Pannag R. Sanketi, Quan Vuong, Thomas Kollar, Benjamin Burchfiel, Russ Tedrake, Dorsa Sadigh, Sergey Levine, Percy Liang, and Chelsea Finn. Open-VLA: An open-source vision-language-action model. In *Proceedings of the 8th Conference on Robot Learning*, volume 270 of *Proceedings of Machine Learning Research*, pp. 2679–2713. PMLR, 2025.
- Qixiu Li, Yaobo Liang, Zeyu Wang, Lin Luo, Xi Chen, Mozheng Liao, Fangyun Wei, Yu Deng, Sicheng Xu, Yizhong Zhang, et al. Cogact: A foundational vision-language-action model for synergizing cognition and action in robotic manipulation. *arXiv preprint arXiv:2411.19650*, 2024.
- Sixu Lin, Junliang Chen, Huaiyuan Xu, Zhuohao Li, Guangming Wang, Yixiong Jing, Sheng Xu, Runyi Zhao, Brian Sheil, Lap-Pui Chau, and Guiliang Liu. RoboFlow4D: A lightweight flow world model toward real-time flow-guided robotic manipulation. In *Proceedings of the 43rd International Conference on Machine Learning*, 2026. URL <https://openreview.net/forum?id=fh6XBnjFlv>.
- Dekun Lu, Wei Gao, and Kui Jia. ImaginationPolicy: Towards generalizable, precise and reliable end-to-end policy for robotic manipulation. *arXiv preprint arXiv:2509.20841*, 2025. URL <https://arxiv.org/abs/2509.20841>.
- Open X-Embodiment Collaboration. Open X-embodiment: Robotic learning datasets and RTX models. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 6892–6903, 2024. doi: 10.1109/ICRA57147.2024.10611477. URL <https://doi.org/10.1109/ICRA57147.2024.10611477>.
- Delin Qu, Haoming Song, Qizhi Chen, Yuanqi Yao, Xinyi Ye, Yan Ding, Zhigang Wang, JiaYuan Gu, Bin Zhao, Dong Wang, et al. Spatialvla: Exploring spatial representations for visual-language-action model. *arXiv preprint arXiv:2501.15830*, 2025.
- Qwen Team. Qwen-robotmanip technical report: Alignment unlocks scale for robotic manipulation foundation models, 2026.
- Mustafa Shukor, Dana Aubakirova, Francesco Capuano, Pepijn Kooijmans, Steven Palma, Adil Zouitine, Michel Aractingi, Caroline Pascal, Martino Russi, Andres Marafioti, Simon Alibert, Matthieu Cord, Thomas Wolf, and Remi Cadene. Smolvla: A vision-language-action model for affordable and efficient robotics. *arXiv preprint arXiv:2506.01844*, 2025.
- Weiliang Tang, Jia-Hui Pan, Wei Zhan, Jianshu Zhou, Huaxiu Yao, Yun-Hui Liu, Masayoshi Tomizuka, Mingyu Ding, and Chi-Wing Fu. Embodiment-agnostic action planning via object-part scene flow. In *2025 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 2086–2093, 2025. doi: 10.1109/ICRA55743.2025.11127873. URL <https://doi.org/10.1109/ICRA55743.2025.11127873>.
- Lirui Wang, Xinlei Chen, Jialiang Zhao, and Kaiming He. Scaling proprioceptive-visual learning with heterogeneous pre-trained transformers. In *Advances in Neural Information Processing*

- Systems*, volume 37, pp. 124420–124450. Curran Associates, Inc., 2024. doi: 10.52202/079017-3952. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/e0f393e7980a24fd12fa6f15adfa25fb-Abstract-Conference.html.
- Mengda Xu, Zhenjia Xu, Yinghao Xu, Cheng Chi, Gordon Wetzstein, Manuela Veloso, and Shuran Song. Flow as the cross-domain manipulation interface. In *Proceedings of the 8th Conference on Robot Learning*, volume 270 of *Proceedings of Machine Learning Research*, pp. 2475–2499. PMLR, 2025. URL <https://proceedings.mlr.press/v270/xu25a.html>.
- Yuyin Yang, Zetao Cai, Yang Tian, Jia Zeng, and Jiangmiao Pang. Gripper pose and object pointflow as interfaces for robotic bimanual manipulation. In *Proceedings of Robotics: Science and Systems*, Los Angeles, CA, USA, June 2025. doi: 10.15607/RSS.2025.XXI.160. URL <https://doi.org/10.15607/RSS.2025.XXI.160>.
- Zhao-Heng Yin, Sherry Yang, and Pieter Abbeel. Object-centric 3D motion field for robot learning from human videos. In *Advances in Neural Information Processing Systems*, volume 38, pp. 55923–55943. Curran Associates, Inc., 2025. doi: 10.52202/085713-1872. URL https://proceedings.neurips.cc/paper_files/paper/2025/hash/50f91c82158baed92e9b7e5491a6d997-Abstract-Conference.html.
- Chengbo Yuan, Chuan Wen, Tong Zhang, and Yang Gao. General flow as foundation affordance for scalable robot learning. In *Proceedings of the 8th Conference on Robot Learning*, volume 270 of *Proceedings of Machine Learning Research*, pp. 1541–1566. PMLR, 2025. URL <https://proceedings.mlr.press/v270/yuan25a.html>.
- Jiahui Zhang, Yurui Chen, Yueming Xu, Ze Huang, Yanpeng Zhou, Yujie Yuan, Xinyue Cai, Guowei Huang, Xingyue Quan, Hang Xu, and Li Zhang. 4d-vla: Spatiotemporal vision-language-action pretraining with cross-scene calibration. *arXiv preprint arXiv:2506.22242*, 2025.
- Jianing Zhang, Chenhao Zheng, Yajun Yang, Max Argus, Rustin Soraki, Winson Han, Taira Anderson, Chun-Liang Li, Shuo Liu, Jiafei Duan, Zhongzheng Ren, Jieyu Zhang, and Ranjay Krishna. MolmoMotion: Forecasting point trajectories in 3D with language instruction. *arXiv preprint arXiv:2606.18558*, 2026. URL <https://arxiv.org/abs/2606.18558>.
- Jinliang Zheng, Jianxiong Li, Dongxiu Liu, Yinan Zheng, Zhihao Wang, Zhonghong Ou, Yu Liu, Jingjing Liu, Ya-Qin Zhang, and Xianyu Zhan. Universal actions for enhanced embodied foundation models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 22508–22519, June 2025. doi: 10.1109/CVPR52734.2025.02096. URL https://openaccess.thecvf.com/content/CVPR2025/html/Zheng_Universal_Actions_for_Enhanced_Embodied_Foundation_Models_CVPR_2025_paper.html.
- Jinliang Zheng, Jianxiong Li, Zhihao Wang, Dongxiu Liu, Xirui Kang, Yuchun Feng, Yinan Zheng, Jiayin Zou, Yilun Chen, Jia Zeng, Tai Wang, Ya-Qin Zhang, Jingjing Liu, and Xianyu Zhan. X-VLA: Soft-prompted transformer as scalable cross-embodiment vision-language-action model. In *The Fourteenth International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=kt51kZH4aG>.
- Ruijie Zheng, Yongyuan Liang, Shuaiyi Huang, Jianfeng Gao, Hal Daumé III, Andrey Kolobov, Furong Huang, and Jianwei Yang. Tracevla: Visual trace prompting enhances spatial-temporal awareness for generalist robotic policies. *arXiv preprint arXiv:2412.10345*, 2024.
- Hongyan Zhi, Peihao Chen, Siyuan Zhou, Yubo Dong, Quanxi Wu, Lei Han, and Minghui Tan. 3DFlowAction: Learning cross-embodiment manipulation from 3D flow world model. *arXiv preprint arXiv:2506.06199*, 2025. URL <https://arxiv.org/abs/2506.06199>.

A DATA CONSTRUCTION DETAILS

This appendix expands the interaction-stage segmentation and 3D trajectory extraction used to construct task-focused point-flow supervision for PointPilot. We briefly summarize simulation processing and focus on gripper-event detection and geometric reconstruction for our self-collected real-robot recordings. The output consists of stage-aligned entity poses and geometry, together with their temporal correspondences, validity information, and reconstruction status.

A.1 STAGE SEGMENTATION

Gripper opening and closing events provide the primary temporal anchors for segmentation. Simulation supplies recorded gripper-state changes; for our real-robot recordings, we recover the corresponding events from the continuous gripper-width signal. We first apply median smoothing and compute the signed numerical gradient with respect to the recorded timestamps. Let s_i denote this gradient at sample i . The automatic event threshold is

$$\tau_s = \max(\tau_{\min}, Q_{0.9}(\{|s_i|\}_t)), \quad (8)$$

where $Q_{0.9}$ denotes the 90th percentile. Contiguous intervals exceeding this magnitude threshold and satisfying a minimum duration in source frames form candidate events. Their direction is determined by the median signed slope: negative for closing and positive for opening.

Nearby events with the same direction are merged. A candidate is accepted only if it agrees with the current open/closed state, reaches the opposite-state plateau, and produces a sufficient change in width. Accepted closing events place a boundary at the end of closure, whereas accepted opening events place one at the onset of opening. The resulting intervals indicate candidate grasp states rather than independently certifying object attachment. Bimanual intervals are synchronized while retaining per-arm information. Simulation additionally uses contact and moving-part cues for interactions not adequately captured by gripper events, such as drawer manipulation. Stage-level language annotations are obtained through a separate offline procedure and associated with the resulting intervals.

A.2 TRAJECTORY EXTRACTION

For simulated demonstrations, we associate recorded object, moving-part, and end-effector (EEF) poses with the corresponding entities and stage intervals, as described in the main paper. Each pose sequence is paired with its entity geometry in a consistent reference frame. We focus below on real-robot reconstruction, where object trajectories must be recovered from visual observations and robot motion rather than directly read from simulator states.

Depth Calibration and Camera Geometry. For our self-collected real-robot recordings, we estimate depth with ViPE (Huang et al., 2025a) on sparse frames associated with grasped-object stages. These depth estimates are used for offline geometric reconstruction, not as inputs to PointPilot at inference. Reconstruction uses the supplied fixed-camera intrinsics and robot-to-camera transforms rather than ViPE’s estimated camera parameters. Estimated depth can have substantial scale and offset errors relative to the robot reference. Even with a correct object mask, directly back-projecting this depth can misplace the object relative to the gripper and distort its reconstructed size. These errors affect the object-to-EEF transform used to propagate object motion. We therefore calibrate the estimated depth against recorded EEF positions before reconstructing the visual anchor.

To construct calibration references, we use the supplied camera calibration to express recorded EEF positions in the camera coordinate frame. Their optical-axis coordinates z_i^E provide the robot-derived depth references. Using the camera intrinsics, we project each reference point into the corresponding depth image at (u_i, v_i) . We then compute d_i , the median of finite, positive estimated depths in a 15×15 pixel patch centered on this projection. This gives a pair (d_i, z_i^E) linking an image-based depth estimate to the robot reference. Usable pairs from both arms and multiple sampled times are pooled to estimate a shared correction.

We model the systematic discrepancy as $z_i^E \approx \alpha d_i + \beta$, where $\alpha > 0$ adjusts depth scale and β adjusts offset. RANSAC identifies a positive-scale relation supported by multiple observations

and rejects pairs with large residuals. Least-squares fitting on the current inliers then refines the parameters, with the inlier set updated from the residuals. The resulting correction is applied to the corresponding depth maps:

$$D_{\text{cal}}(u, v) = \alpha D_{\text{est}}(u, v) + \beta. \quad (9)$$

The same fitted parameters are applied to all associated frames, rather than fitting each pixel or frame independently. Although the references are sampled near the EEf projections, the correction is applied to the depth maps used for object reconstruction, not only to those local patches.

We accept a fit only when it has sufficient supporting observations, adequate depth variation and correlation, and a scale within the allowed range. If these references are insufficient for a supported fit, the designated failure path retains the supplied camera geometry and uncalibrated depth with an explicit calibration-failure status. The EEf frame origin need not lie on a visible gripper surface, so the sampled patch provides an approximate correspondence rather than an exact depth measurement of that point. The calibration therefore constrains local consistency with the robot reference without establishing scene-wide depth ground truth or an object-trajectory error bound.

Visual Anchor Selection. For a grasped-object stage, we search for a visual anchor within the candidate grasp interval. Adapting the visual-grounding components of MolmoMotion (Zhang et al., 2026), we use MolmoPoint (Clark et al., 2026) to obtain image-plane location cues and SAM 3 (Carion et al., 2025) to generate candidate masks. Localization specifies the manipulated object and the corresponding arm. MolmoPoint processes each candidate image as a short repeated-frame clip; its locations guide selection from text-conditioned SAM 3 masks. When no suitable text-conditioned mask is matched, a small box around a predicted location provides an additional segmentation prompt. These components localize candidate anchors rather than track the object throughout the stage.

We evaluate up to five candidate frames and separate acceptance from ranking. A supported candidate must meet foreground-area and mask-to-EEf proximity thresholds. A stage requires at least $\min(3, N)$ supported candidates, where N is the number examined. Among supported candidates, we select the highest-scoring anchor using

$$Q = 0.25Q_{\text{area}} + 0.40Q_{\text{depth}} + 0.15Q_{\text{boundary}} + 0.20Q_{\text{proximity}}, \quad (10)$$

where the terms quantify mask area, valid-depth coverage, image-boundary margin, and proximity to the projected EEf of the corresponding arm. Support across candidate frames is used as a consistency check; only the selected anchor is used to recover object geometry. We do not fuse multiple anchor poses or interpret the score as a calibrated probability of reconstruction accuracy.

Object Geometry and Motion Reconstruction. Following the reconstruction convention in the main paper, we express reconstructed object geometry in the supplied fixed-camera frame C . At the selected anchor time a , valid masked depth pixels are back-projected using the supplied intrinsics:

$$\mathbf{p}_a^C(u, v) = D_a(u, v) \begin{bmatrix} (u - c_x)/f_x \\ (v - c_y)/f_y \\ 1 \end{bmatrix}, \quad (11)$$

where D_a is the accepted depth map, using the affine correction when available. We cap the point cloud at 30,000 samples, remove the most distant 10% relative to its coordinate-wise median, and compute a right-handed PCA frame. The 2nd and 98th percentiles along each axis determine a robust box center and extent, with a minimum extent for nearly degenerate dimensions. The resulting anchor pose $\hat{T}_{O_a}^C$ and oriented bounding box (OBB) approximate the visible object region rather than a complete CAD-aligned object model.

We recover temporal object motion through the fixed relative transform described in the main paper:

$$\hat{T}_O^E = (T_{E_a}^C)^{-1} \hat{T}_{O_a}^C, \quad \hat{T}_{O_t}^C = T_{E_t}^C \hat{T}_O^E, \quad t \in \mathcal{I}_{\text{coupled}}. \quad (12)$$

The same transformation propagates the object-local OBB vertices. Thus, visual reconstruction estimates the anchor geometry once, while recorded EEF poses provide the subsequent motion. This construction assumes approximately rigid attachment without appreciable slip within $\mathcal{I}_{\text{coupled}}$; the existence of a stage alone does not establish that assumption. Outside the reconstruction interval, including after release, object poses and vertices are marked unavailable rather than propagated with the former gripper attachment. Errors in anchor geometry, camera alignment, and the attachment assumption can still affect the reconstructed trajectory.

Gripper Geometry and Fallback Handling. We construct a sequence of gripper OBBs from URDF collision geometry, recorded joint states, and EEF poses. The joint states determine the gripper configuration, while the EEF poses place the geometry in the scene. Before combining these sources, we check that the logged EEF poses are consistent with forward-kinematic flange poses under the supplied reference-frame mapping. The resulting bounding boxes describe the gripper’s motion and opening without requiring image-based reconstruction.

When object localization or geometric reconstruction fails to produce an accepted object trajectory, we fall back to the corresponding gripper trajectory and OBB derived from recorded EEF poses and robot geometry. Fallback cases are explicitly flagged and treated as gripper geometry rather than successfully reconstructed object motion.

Finally, stage intervals, language annotations, and geometric outputs are associated through source-frame indices and timestamps. Reconstruction validity and fallback provenance are retained separately. The stored object masks correspond to selected anchors rather than dense per-frame segmentation, and the sparse geometry timeline is not assumed to be uniformly sampled. These records provide pose sequences and validity information for constructing PointPilot’s point-flow supervision for task-focused objects. The prediction targets are future 3D point displacements relative to their current positions, as defined in the main paper.